7: Kubernetes Services

In previous chapters, you've deployed applications to Kubernetes and seen how controllers add self-healing, scaling and rollouts. Despite all of this, **Pods are still unreliable** and you should not connect directly to them. This is where *Services* come to the rescue providing stable and reliable networking for a set of unreliable Pods.

The chapter is divided as follows:

- Setting the scene
- Theory
- Hands-on

Setting the scene

When Pods fail, they get replaced by new ones with new IPs. Scaling-up introduces new Pods with new IP addresses. Scaling down removes Pods. Rolling updates also replace existing Pods with new ones with new IPs. This creates massive *IP churn* and demonstrates why you should never connect directly to any particular Pod.

You also need to know 3 fundamental things about Kubernetes Services.

First, when talking about *Services* with a capital "S", we're talking about the Service object in Kubernetes that provides stable networking for Pods. Just like a *Pod*, *ReplicaSet*, or *Deployment*, a Kubernetes *Service* is a REST object in the API that you define in a manifest and post to the API server.

Second, every Service gets its own **stable IP address**, its own **stable DNS name**, and its own **stable port**.

Third, Services use *labels* and *selectors* to dynamically select the Pods to send traffic to.

Service Theory

Figure 7.1 shows a simple application managed by a Deployment controller. There's a client (which could be another microservice) that needs a reliable network endpoint to access the Pods. Remember, it's a bad idea to talk directly to individual Pods because they can disappear at any point via scaling operations, rollouts and rollbacks, and failures.

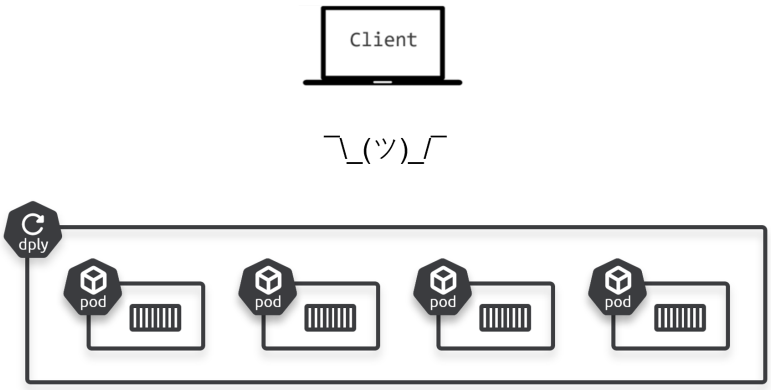

Figure 7.1

Figure 7.2 shows the same application with a Service thrown into the mix. The Service fronts the Pods with a stable IP, DNS name, and port. It also load-balances traffic to Pods with the right labels.

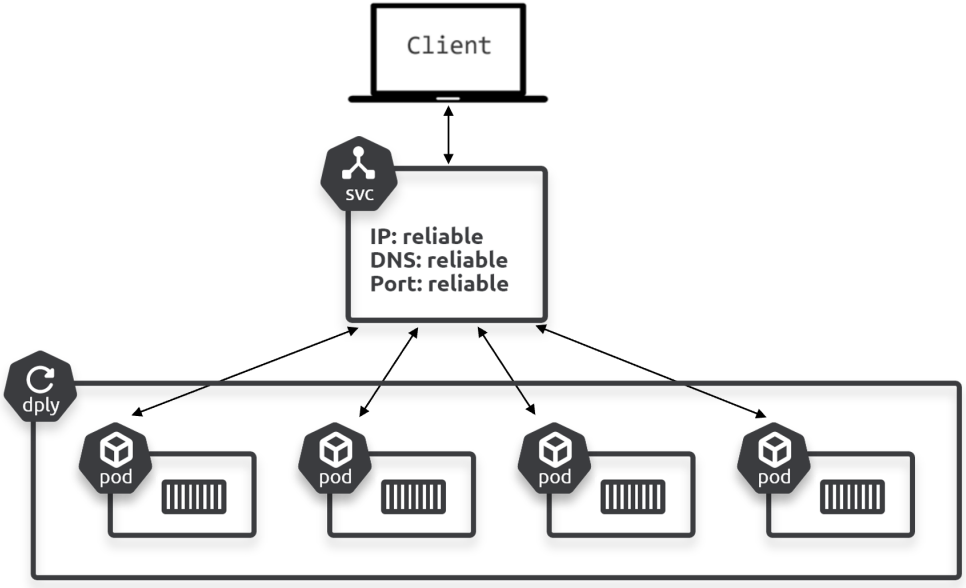

Figure 7.2

With a Service in place, the Pods can scale up and down, they can fail, and they can be updated and rolled back. And clients will continue to access them without interruption. This is because the Service is observing the changes and updating its list of healthy Pods. ***But it never changes its stable IP, DNS, and port***.

Think of Services as having a static front-end and a dynamic back-end. The front-end, consisting of the IP, DNS name, and port, never changes. The back-end, comprising the list of healthy Pods, can be constantly changing.

Labels and loose coupling

Services are loosely coupled with Pods via *labels* and *selectors*. This is the same technology that loosely couples Deployments to Pods and is key to the flexibility of Kubernetes. Figure 7.3 shows an example where 3 Pods are labelled as `zone=prod` and `version=1`, and the Service has a *selector* that matches.

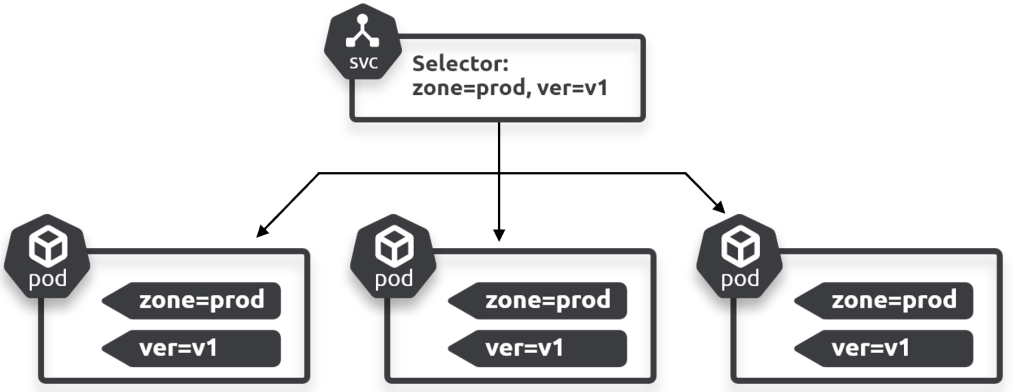

Figure 7.3

In Figure 7.3, the Service is providing stable networking to all three Pods – you send requests to the Service and it forwards them to the Pods. It also provides basic load-balancing.

For a Service to send traffic to a Pod, the Pod needs every label the Service is selecting on. It can also have additional labels the Service isn't looking for. If that's confusing, the examples in Figures 7.4 and 7.5 will help.

Figure 7.4 shows an example where none of the Pods match. This is because the Service is looking for Pods with both labels. The logic is a Boolean `AND`.

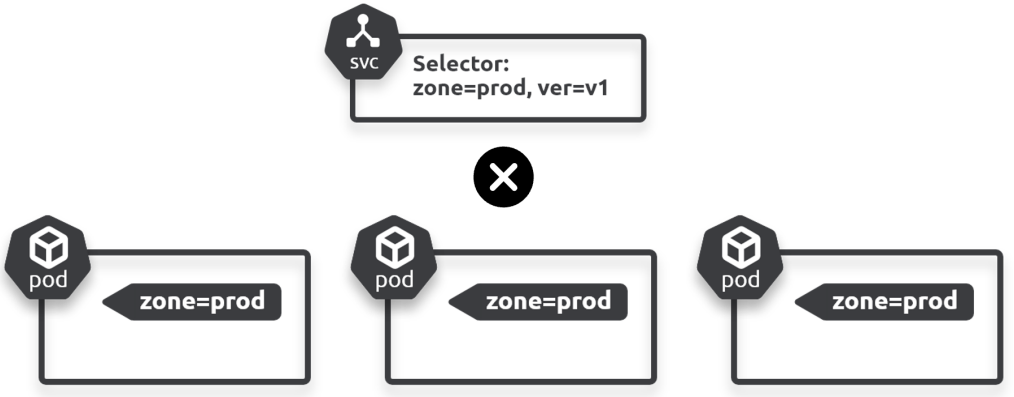

Figure 7.4

Figure 7.5 shows an example that does work. This time the Pods have all the labels the Service is selecting on. It makes no difference if the Pods have additional labels. The Service is looking for Pods with two specific labels,

it finds them, and it ignores the fact they have additional labels.

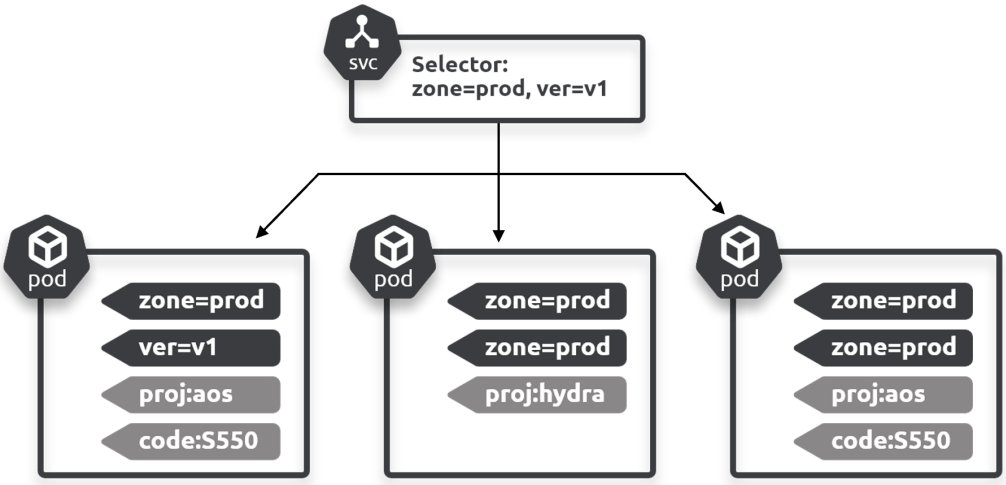

Figure 7.5

The following excerpts, from a Service YAML and Deployment YAML, show how *selectors* and *labels* work.

svc.yml

```
apiVersion: v1
kind: Service
metadata:
  name: hello-svc
spec:
  ports:
  - port: 8080
  selector:
    app: hello-world   <<== Send to Pods with these labels
    env: tkb           <<== Send to Pods with these labels
```

deploy.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  <Snip>
  template:
    metadata:
      labels:
        app: hello-world     <<==  Pod labels
```

```
      env: tkb            <<==  Pod labels
  spec:
    containers:
<Snip>
```

In the example, the Service is selecting on Pods with the `app=hello-world` and `book=tkb` labels. The Deployment has a Pod template with the same two labels. This means any Pods it deploys will match the Service's selector and receive traffic from it. This loose coupling is how Services know which Pods to send traffic to.

Services and Endpoint objects

As Pods come-and-go (scaling, failures, rollouts etc.), the Service dynamically updates its list of healthy matching Pods. It does this through a combination of label selection and a construct called an *Endpoints* object.

Every time you create a Service, Kubernetes automatically creates an associated *Endpoints* object. The Endpoints object is used to store a dynamic list of healthy Pods matching the Service's label selector.

It works like this…

Kubernetes is constantly evaluating the Service's label selector against all healthy Pods on the cluster. Any new Pods that match the selector get added to the Endpoints object, whereas any Pods that disappear get removed. This means the Endpoints object is always up to date.

When sending traffic to Pods via a Service, the cluster's internal DNS to resolves the Service name to an IP address. It then sends the traffic to this stable IP address and the traffic gets routed to one of the Pods in the Endpoints list. However, a *Kubernetes-native* application (that's a fancy way of saying an application that understands Kubernetes and can query the Kubernetes API) can query the Endpoints API directly, bypassing the DNS lookup and use of the Service's IP.

> **Note:** Recent versions of Kubernetes are replacing *Endpoints* objects with more efficient *Endpoint slices*. The functionality is identical, but *Endpoint slices* are higher performance and more efficient.

Now that you know the fundamentals, let's look at some use-cases.

Accessing Services from inside the cluster

Kubernetes supports several *types* of Service. The default type is *ClusterIP*.

A *ClusterIP* Service has a stable virtual IP address that is **only accessible from inside the cluster**. We call this a "ClusterIP". It's programmed into the network fabric and guaranteed to be stable for the life of the Service. *Programmed into the network fabric* is fancy way of saying the network *just knows about it* and you don't need to bother with the details.

Anyway, the ClusterIP is registered against the name of the Service in the cluster's internal DNS service. All Pods in the cluster are pre-programmed to use the cluster's DNS service, meaning all Pods can convert Service names to ClusterIPs.

Let's look at a simple example.

Creating a new Service called "magic-sandbox" will dynamically assign a stable ClusterIP. This name and ClusterIP are automatically registered with the cluster's DNS service. These are all guaranteed to be long-lived

and stable. As all Pods in the cluster send service discovery requests to the internal DNS, they can all resolve "magic-sandbox" to the ClusterIP. iptables or IPVS rules are distributed across the cluster to ensure traffic sent to the ClusterIP gets routed to matching Pods.

Net result… if a Pod (application microservice) knows the name of a Service, it can resolve that to a ClusterIP address and connect to the Pods behind it.

This only works for Pods and other objects on the cluster, as it requires access to the cluster's DNS service. It does not work outside of the cluster.

Accessing Services from outside the cluster

Kubernetes has two types of Service for requests originating from outside the cluster.

- NodePort
- LoadBalancer

NodePort Services build on top of the ClusterIP type and enable external access via a dedicated port on every cluster node. We call this port the "NodePort".

You already know the default Service type is ClusterIP, and it registers a DNS name, virtual IP, and port with the cluster's DNS. NodePort Services build on this by adding a *NodePort* that can be used to reach the Service from outside the cluster.

The following YAML shows a NodePort Service called "magic-sandbox".

```
apiVersion: v1
kind: Service
metadata:
  name: magic-sandbox
spec:
  type: NodePort
  ports:
  - port: 8080
    nodePort: 30050
  selector:
    app: hello-world
```

Pods on the cluster can access this Service by the name "magic-sandbox" on port `8080`. Clients connecting from outside the cluster can send traffic to any cluster node on port `30050`.

Figure 7.6 shows a NodePort Service where 3 Pods are exposed externally on port `30050` on every cluster node. In step 1, an external client hits **node2** on port `30050`. In step 2 it's redirected to the Service object. Step 3 shows the associated Endpoint object with an always-up-to-date list of Pods matching the label selector. Step 4 shows the client being directed to a healthy Pod on **node1**.

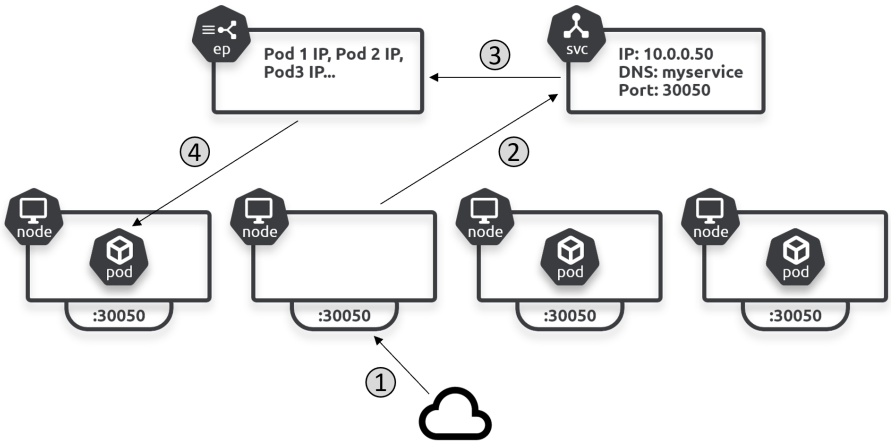

Figure 7.6

The Service could just as easily have directed the client to a Pod on node 3 or 4. In fact, future requests may go to other Pods as the Service performs basic load-balancing.

LoadBalancer Services make external access even easier by integrating with an internet-facing load-balancer on your underlying cloud platform. You get a high-performance highly-available public IP or DNS name that you can access the Service from. You can even register friendly DNS names to make access even simpler – you don't need to know cluster node names or IPs.

You'll create and use a LoadBalancer Service in the hands-on section later. They only work on clouds that support them.

Service discovery

The book has an entire chapter dedicated to a service discovery deep dive, so this section will be brief.

Kubernetes implements Service discovery in a couple of ways:

- DNS (preferred)
- Environment variables (definitely not preferred)

Kubernetes clusters run an internal DNS service that is the centre of service discovery. Service names are automatically registered with the cluster DNS, and every Pod and container is pre-configured to use the cluster DNS. This means every Pod/container can resolve every Service name to a ClusterIP and connect to the Pods behind it.

The alternative form of service discovery is through environment variables. Every Pod gets a set of environment variables that resolve Services currently on the cluster. However, they cannot learn about new Services added after the Pod they are in was created. This is a major reason DNS is the preferred method.

See the service discovery chapter for a deeper dive.

Summary of Service theory

The front-end of a Service provides an immutable IP, DNS name and port that is guaranteed not to change for the entire life of the Service. The back-end of a Service uses labels and selectors to load-balance traffic across a potentially dynamic set of application Pods.

Hands-on with Services

Let's put the theory to the test.

There are two ways to create Services.

- The imperative way (not recommended)
- The declarative way

You'll test Services for internal and external access, and take a closer look at Endpoints objects.

To follow along, you'll need the lab files from the book's GitHub repo. You can clone the repo with the following command, or you can manually copy the file contents into files on your local machine.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
Cloning into 'TheK8sBook'...
```

The imperative way

The imperative way is not the Kubernetes way. It moves your cluster and apps out of sync with your YAML files.

Use `kubectl` to declaratively deploy the following Deployment (later steps will be done imperatively). Be sure to run the command from within the `services` folder.

```
$ kubectl apply -f deploy.yml
deployment.apps/svc-test created
```

Now the "svc-test" Deployment is running, it's time to *imperatively* deploy a Service for it.

The following command imperatively creates a new Service that will provide networking and load-balancing for the Pods created and managed by the `svc-test` Deployment.

```
$ kubectl expose deployment svc-test --type=NodePort
service/hello-svc exposed
```

Run a `kubectl get` to ensure it was created.

```
$ kubectl get svc -o wide
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE   SELECTOR
kubernetes   ClusterIP   10.43.0.1      <none>        443/TCP          19d   <none>
svc-test     NodePort    10.43.48.145   <none>        8080:30013/TCP   10s   chapter=services
```

The first line is a system Service that exposes the Kubernetes API on the cluster. Your Service is on the second line, and you can see the imperative command was clever enough to read the Deployment spec and configure the correct label selector and container port. It also gave the Service the same name as the Deployment.

You can get more detailed info with the `kubectl describe` command.

```
$ kubectl describe svc svc-test
Name:                     svc-test
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 chapter=services
Type:                     NodePort
IP:                       10.43.48.145
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  30013/TCP
Endpoints:                10.42.0.116:8080,10.42.0.117:8080,10.42.1.118:8080 + 2 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Events:                   <none>```
```

Some interesting values in the output include:

- `Selector` is the list of labels the Service looks for when building its list of Pods to send traffic to
- `IP` is the permanent internal ClusterIP (VIP) of the Service
- `Port` is the port the Service listens on inside the cluster
- `TargetPort` is the port the application is listening on
- `NodePort` is the cluster-wide port that can be used for external access
- `Endpoints` is the dynamic list of healthy Pod IPs that match the selector

Now that you know the cluster-wide NodePort (`30013`), you can open a web browser and access the app. To do this, you need to know the IP address of at least one of your cluster nodes, and you need to be able to reach it from your browser – e.g. a publicly routable IP if you're accessing via the internet.

Figure 7.7 shows a web browser accessing a node with an IP address of `54.246.255.52` on the cluster-wide NodePort `30013`.

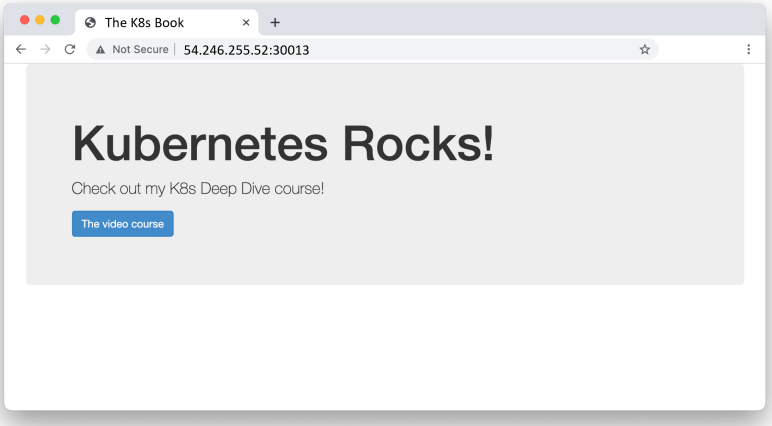

Figure 7.7

The app you've deployed is a simple web app. It's built to listen on port `8080`, and you've configured a Kubernetes *Service* to map port `30013` on every cluster node back to port `8080` on the app. By default, cluster-wide NodePorts are between 30,000 - 32,767. In this example it was dynamically assigned, but you can also explicitly choose a port if using a YAML file.

Coming up next, you'll see how to do the same thing the proper way – the declarative way. To do that, you need to clean up by deleting the Service you just created. You can do this with the following `kubectl delete svc` command.

```
$ kubectl delete svc svc-test
service "svc-test" deleted
```

The declarative way

Time to do things the proper way… the Kubernetes way.

A Service manifest file

You'll use the following resource manifest to deploy the same *Service* you deployed in the previous section. However, this time you'll specify a value for the cluster-wide NodePort.

```
apiVersion: v1
kind: Service
metadata:
  name: svc-test
spec:
  type: NodePort
  ports:
  - port: 8080
    nodePort: 30001
    targetPort: 8080
    protocol: TCP
  selector:
    chapter: services
```

Let's step through it.

Services are mature objects defined in the `v1` core API group (`apiVersion`).

The `kind` field tells Kubernetes you're defining a Service object.

The `metadata` block defines a name for the Service. You can also apply labels and annotations here. Any labels you add here are used to identify the Service and are not related to selecting Pods.

The `spec` section is where you actually define the Service. This example is telling Kubernetes to deploy a `NodePort` Service. The `port` value tells Kubernetes to listen internally on port `8080`, and the `NodePort` value tells it to listen externally on `30001`. The `targetPort` value is part of the back-end configuration and tells Kubernetes to forward traffic to the application Pods on port `8080`. Then you're explicitly telling it to use TCP (default). Finally, `spec.selector` tells the Service to send traffic to all healthy Pods on the cluster with the `chapter=services` label.

Deploy it with the following command.

```
$ kubectl apply -f svc.yml
service/svc-test created
```

Introspecting Services

Now the Service is deployed, you can inspect it with the usual `kubectl get` and `kubectl describe` commands.

```
$ kubectl get svc svc-test
NAME        TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)         AGE
hello-svc   NodePort   100.70.40.2     <none>        8080:30001/TCP  8s
```

The Service is up and exposed via every cluster node on port `30001`. This means you can point a web browser to the name or IP of any node on that port and reach the Service. You'll need to use the IP address of a node you can reach, and you'll need to ensure any firewall and security rules allow the traffic to flow.

Endpoints objects

Earlier in the chapter, you learned every Service gets its own *Endpoints* object with the same name as the Service. This holds a list of all the Pods the Service matches and is dynamically updated as matching Pods come and go. You can see Endpoints, and the newer EndpointSlices, with the normal `kubectl` commands.

The following examples show EndpointSlices, but you can run the same commands for Endpoints.

```
$ kubectl get endpointslices
NAME             ADDRESSTYPE   PORTS   ENDPOINTS                                 AGE
svc-test-sbhbj   IPv4          8080    10.42.1.119,10.42.0.117,10.42.1.120...    6m38s

$ Kubectl describe endpointslice svc-test-sbhbj
Name:         svc-test-sbhbj
Namespace:    default
Labels:       endpointslice.kubernetes.io/managed-by=endpointslice-controller.k8s.io
              kubernetes.io/service-name=svc-test
Annotations:  endpoints.kubernetes.io/last-change-trigger-time: 2021-02-05T20:01:31Z
AddressType:  IPv4
Ports:
  Name     Port  Protocol
  ----     ----  --------
  <unset>  8080  TCP
Endpoints:
  - Addresses:  10.42.1.119
    Conditions:
      Ready:    true
    Hostname:   <unset>
    TargetRef:  Pod/svc-test-84db6ff656-wd5w7
    Topology:   kubernetes.io/hostname=k3d-gsk-book-server-0
  - Addresses:  10.42.0.117
<Snip>
Events:         <none>
```

The full output of the `kubectl describe` command has a block for each healthy Pod containing useful info.

LoadBalancer Services

Now for the best type of Service. And it's also the easiest.

If your cluster is on a cloud platform, deploying a Service with `type=LoadBalancer` will provision one of your cloud's internet-facing load-balancers and configure it to send traffic to your Service. It's a beautiful thing.

The following YAML is from the `lb.yml` file. It defines a new Service called "clou-lb" that will provision a cloud load-balancer listening on port `9000` and forwarding traffic on port `8080` to all Pods with the `chapter=services` label. Basically, a cloud load-balancer sending traffic to the Pods you already deployed through the "svc-test" Deployment.

```
apiVersion: v1
kind: Service
metadata:
  name: cloud-lb
spec:
  type: LoadBalancer
  ports:
  - port: 9000
    targetPort: 8080
  selector:
    chapter: services
```

Deploy it with the following command.

```
$ kubectl apply -f lb.yml
service/cloud-lb created
```

Now list it.

```
$ kubectl get svc --watch
NAME         TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
cloud-lb     LoadBalancer   10.43.128.113   172.21.0.4    9000:32688/TCP   47s
<Snip>
```

The `EXTERNAL-IP` column shows the public address assigned to the Service by your cloud. On some cloud platforms it may be a DNS name instead of an IP, and it may take a minute or two to populate. The delay is while Kubernetes works with your cloud platform to provision and configure the load-balancer.

Copy the value in the `EXTERNAL-IP` column and paste it into your browser with port `9000` to get to the app.

Congratulations, you just configured a high performance highly-available internet facing load-balancer to front your Service.

Clean-up

Clean-up the lab with the following command. These will delete the Deployment and Services. Endpoints and EndpointSlices are automatically deleted with their Service.

```
$ kubectl delete -f deploy.yml -f svc.yml -f lb.yml
deployment.apps "svc-test" deleted
service "svc-test" deleted
service "cloud-lb" deleted
```

Chapter Summary

In this chapter, you learned that *Services* bring stable and reliable networking to apps deployed on Kubernetes. On the front end, they provide a stable DNS name that's automatically registered with the cluster DNS, as well as a stable virtual IP. On the back end, they load-balance traffic across a dynamic set of Pods that match a label selector. They also let you expose elements of your application to the outside world, including integrating with cloud load-balancers.

Services are first-class objects in the Kubernetes API and should be managed declaratively through version-controlled YAML manifest files.